

Chapter 1: Understanding Python

Easy-to-Understand Numerical Analysis Using Python

Prof. See Jo Kim

Gyeongbuk National University
School of Electronics and Mechanics
Major of Robotics Engineering

January 24, 2026

Table of Contents

1 (1.1) Features of Python

- (1.1.1) Platform Independence
- (1.1.2) Interpreter Language
- (1.1.3) Blocks and Line Breaks
- (1.1.4) Object-Oriented Programming Support
- (1.1.5) Dynamic Typing
- (1.1.6) NumPy Library

2 (1.2) Variable Declaration

- (1.2.1) Data Types, Variable Declaration and Conversion
- (1.2.2) Python Reserved Keywords

3 (1.3) Arrays and Matrices

- (1.3.1) Arrays Using Lists
- (1.3.2) List Slicing
- (1.3.3) NumPy and Arrays
- (1.3.4) Matrices and Arrays

4 (1.4) Tuples

Chapter 1: Understanding Python

Overview

Python is one of the most intuitive and flexible tools among modern programming languages, especially establishing itself as a powerful tool in engineering calculations such as numerical analysis.

Key Characteristics:

- Platform independence
- Concise syntax
- Dynamic typing
- Interpreter-based execution
- Extensive library ecosystem

Why Python for Numerical Analysis?

- Can perform identical numerical calculations across various environments with the same code
- Immediate result verification through code writing and execution
- Provides great advantages for learners and researchers

From a Numerical Analysis Perspective:

- Tool for quickly and simply solving engineering problems
- Simplifies repetitive operations with simple syntax
- Users can focus on algorithm implementation
- Efficiently handles entire numerical problem-solving process

Key Libraries for Numerical Analysis:

NumPy Supports high-performance numerical operations including vector and matrix operations

pandas Helps easily manipulate and analyze complex datasets

Matplotlib Provides tools for intuitively understanding results through data visualization

SciPy Provides advanced computation functions needed for mathematical, scientific, and engineering problems

Conclusion: Python significantly enhances engineering problem-solving capabilities by providing extensive tools and libraries for easily handling complex numerical analysis problems.

(1.1.1) Platform Independence

Definition

Python code can obtain identical results on various operating systems such as Windows, macOS, and Linux.

Example: Simple file reading code works identically on all operating systems

```
with open('example.txt', 'r') as file:  
    print(file.read())
```

Advantages:

- Write once, run anywhere
- No platform-specific modifications needed
- Ideal for collaborative projects across different systems

(1.1.2) Interpreter Language

Definition

You can write Python code and immediately execute it to check results.

Example: Simple calculation in Python shell

```
print(2 + 3)  
# Output: 5
```

Benefits:

- Rapid prototyping and testing
- Interactive development
- Immediate feedback for learning
- No compilation step required

(1.1.3) Blocks and Line Breaks

Indentation in Python

Python uses indentation to define blocks instead of braces `{}`.

Indentation Rules:

- Code within a block should be indented by 4 spaces or a tab
- Indentation size must be consistent within the same block
- Mixed indentation causes errors
- Use `pass` keyword for empty blocks

Comparison:

- C/C++/Java: Use braces `{}`
- MATLAB: Use `if-else-end` structure
- Python: Use indentation only

Example: Conditional and Loop Blocks

```
n = 5
if n > 0: # Conditional block starts
    print("n is positive")

for i in range(n): # Loop block starts
    print(f"Iteration {i}")
    print("Loop finished")

print("Outside all blocks")
```

Output:

n is positive

Iteration 0

Iteration 1

...

Loop finished

Outside all blocks

(1.1.4) Object-Oriented Programming Support

Defining classes and creating objects:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "makes a sound"

class Dog(Animal):
    def speak(self):
        return "barks"

pet = Dog("Buddy")
print(pet.name, pet.speak())
# Output: Buddy barks
```

(1.1.5) Dynamic Typing

Definition

Variables can be assigned and changed to various types of values without type declaration, making code writing more flexible and simple.

```
number = 10          # Initially assign an integer
print(number)        # Output: 10

number = "ten"       # Later assign a string
print(number)        # Output: ten
```

Advantages:

- More flexible and simpler code
- No explicit type declarations needed
- Quick prototyping

(1.1.6) NumPy Library

Definition

NumPy is a library that enables high-performance numerical calculations in Python, optimized for large multidimensional arrays and matrix operations.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b) # Array addition
# Output: [5 7 9]
```

Key Features:

- High-performance array operations
- Broadcasting capabilities
- Extensive mathematical functions
- Foundation for scientific computing in Python

(1.2) Variable Declaration

Overview

This section explains variable declaration and usage in Python. Python automatically determines data types when values are assigned, reducing the burden of data type declaration and ensuring flexibility in program design.

Key Concepts:

- No explicit type declaration needed
- Data type determined automatically upon value assignment
- Type casting available through functions like `int()`, `float()`, `str()`
- Cannot use reserved keywords as variable names

(1.2.1.1) Data Types and Variables

Python supports dynamic typing without type declaration:

```
x = 10                # Integer
print(x)              # Output: 10

y = 3.14              # Float
print(y)              # Output: 3.14

name = "Alice"        # String
print(name)           # Output: Alice

is_valid = True        # Boolean
print(is_valid)        # Output: True
```

Key Points:

- Even single characters are treated as strings (str)
- Easy conversion between numbers and strings
- Integers have no size limit

(1.2.1.2) Type Casting Examples

Converting between data types:

```
# Float to Integer (truncates decimal)
x = int(3.9)
print(x) # Output: 3

# Integer to Float
y = float(5)
print(y) # Output: 5.0

# Integer to String
z = str(42)
print(z) # Output: "42"

# String to Float
a = "3.14"
b = float(a)
print(b) # Output: 3.14
```

(1.2.1.3) Type Casting Meaning

Definition

Type casting is the process of changing data from one data type to another.

Common Use Cases:

- Converting strings to numbers for calculations
- Converting numbers to strings for formatted output
- Adjusting data types for specific operations

Important Note: Type conversion can result in data loss (e.g., converting float to int truncates decimal part), so caution is needed during the conversion process.

(1.2.2) Python Reserved Keywords

Definition

Reserved keywords are predefined words used to define the language's syntax and structure. These keywords cannot be used as variable or function names.

Python Keywords (33 total):

and	as	assert	break	class
continue	def	del	elif	else
except	exec	finally	for	from
global	if	import	in	is
lambda	nonlocal	not	or	pass
print	raise	return	try	while
with	yield			

Remember: Keywords are case-sensitive!

(1.3) Arrays and Matrices

Overview

This section explains methods for handling arrays and matrices in Python and their applications.

Python Lists vs NumPy Arrays:

- **Lists:** Simple arrays, can store various data types, flexible but slower
- **NumPy Arrays:** High-performance arrays for large-scale data, optimized for mathematical operations

Key Topics:

- Basic array creation using lists
- Advanced array operations using NumPy
- Matrix operations
- Applications in data analysis and scientific computing

(1.3.1) Arrays Using Lists

List Format and Characteristics:

```
# Basic list format
list_name = [element1, element2, element3, ...]

# Examples
colors = ['red', 'green']           # String list
empty_list = []                     # Empty list
mixed_list = ['Hello', 1, 2]        # Mixed types
nested = [['Hello', 'World'], 1, 2] # Nested list
```

Characteristics:

- Can store various data types together
- Dynamic size adjustment
- Treated as row vector by default
- Not optimized for mathematical operations

List Data Access and Operations

```
a = [10, 20, 30, 40]
print(a[0])    # Output: 10 (first element)
print(a[-1])   # Output: 40 (last element)

# Operations
sum = a[0] + a[1]
print(sum)     # Output: 30

# Nested lists
a = [1, 2, ['a', 'b'], ['Life', 'is']]
print(a[2][2][0]) # Output: Life
```

Key Points:

- Indexing starts at 0
- Negative indices access from end
- Nested indexing for multidimensional access

(1.3.2) List Slicing

Slicing Syntax

`list_name[start:end:step]`

- **start:** Starting index (inclusive)
- **end:** Ending index (exclusive)
- **step:** Index interval (default 1)

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(a[2:5])    # [2, 3, 4]
print(a[:4])     # [0, 1, 2, 3]
print(a[5:])     # [5, 6, 7, 8, 9]
print(a[::2])    # [0, 2, 4, 6, 8]
print(a[-3:])    # [7, 8, 9]
print(a[::-1])   # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

(1.3.3) NumPy and Arrays

NumPy provides optimized array operations:

```
import numpy as np

# Create NumPy array
array_np = np.array([1, 2, 3, 4])
print(array_np) # [1 2 3 4]

# 2D array
array_2d = np.array([[1, 2], [3, 4]])
print(array_2d)
# [[1 2]
#   [3 4]]

# Vectorized operation
result = array_np + 10
print(result) # [11 12 13 14]
```

Advantages over lists:

- Memory efficient
- Faster operations
- Vectorized computations

(1.3.4) Matrices and Arrays

Matrix operations using NumPy:

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
C = A @ B
print(C)
# [[19 22]
#  [43 50]]

# Transpose
transpose = A.T
print(transpose)
# [[1 3]
#  [2 4]]
```

Key Operations:

- Matrix multiplication: @ or np.dot()
- Transpose: .T
- Element-wise operations supported

(1.4) Tuples

Definition

Tuples are similar to lists but are immutable (cannot be changed after creation), suitable for maintaining data integrity.

```
# Tuple creation
T = ((10, 20), (30, 40), (50, 60))
print('T=', T)
print('T[0][0]=', T[0][0]) # Output: 10

# Nested tuples
T4 = (1, 2, ('a', 'b', ('Life', 4)))
print(T4[2][2][0]) # Output: Life
```

Key Difference from Lists:

- Lists use square brackets []
- Tuples use parentheses ()
- Tuples are immutable

Tuple Characteristics

Immutability:

```
T = ((10, 20), (30, 40))  
# T[0][0] = 55 # This will raise TypeError
```

Single-element tuples need a comma:

```
T = (55,) # Correct single-element tuple  
print(T[0]) # Output: 55  
  
T = (55) # This is just an integer, not a tuple  
# print(T[0]) # TypeError: 'int' object is not subscriptable
```

When to Use Tuples:

- Data that shouldn't be modified
- Function return multiple values
- Dictionary keys (tuples can be keys, lists cannot)

(1.5) Dictionaries

Definition

A dictionary is a data structure that stores data as key-value pairs, making data search and manipulation very fast and efficient.

```
# Dictionary creation
student = {
    "name": "Alice",
    "age": 25,
    "major": "Engineering"
}

# Value access
print(student["name"]) # Output: Alice

# Value modification
student["age"] = 26

# Adding new key-value pair
student["grade"] = "A"

# Deleting a key
del student["major"]

print(student) # {'name': 'Alice', 'age': 26, 'grade': 'A'}
```

Dictionary Characteristics

Key Features:

- **Keys:** Must be immutable (strings, numbers, tuples)
- **Values:** Can be any data type
- **Speed:** Fast data search and modification
- **Mutable:** Can dynamically add, modify, or delete entries

Common Operations:

- Access: `dict[key]`
- Add/Modify: `dict[key] = value`
- Delete: `del dict[key]`
- Check existence: `key in dict`

Use Cases:

- Storing structured data
- Configuration settings
- Counting occurrences

(1.6) Mathematical Operators and NumPy Comparison

Overview

This section explains the differences between Python's basic mathematical operators and the advanced numerical computation library NumPy.

Operation	Python	NumPy
Addition	<code>a + b</code>	<code>np.add(a, b)</code>
Multiplication	<code>a * b</code>	<code>np.multiply(a, b)</code>
Division	<code>a / b</code>	<code>np.divide(a, b)</code>
Power	<code>a ** b</code>	<code>np.power(a, b)</code>
Square root	-	<code>np.sqrt(a)</code>
Matrix mult.	-	<code>np.dot(a, b)</code> or <code>a @ b</code>

Python lists vs NumPy arrays for large-scale operations:

```
import numpy as np
import time

# Python list operation
a = list(range(1_000_000))
b = list(range(1_000_000))
start = time.time()
c = [a[i] + b[i] for i in range(len(a))]
print("List operation:", time.time() - start)

# NumPy array operation
a_np = np.arange(1_000_000)
b_np = np.arange(1_000_000)
start = time.time()
c_np = a_np + b_np
print("NumPy operation:", time.time() - start)
```

Result: NumPy is significantly faster (often 10-100x) for numerical operations!

(1.7) Iterative Calculations

Overview

This section explains various methods for performing iterative calculations in Python using `for` and `while` loops.

Key Concepts:

- **for loops:** Iterate over sequences or ranges
- **while loops:** Continue while condition is true
- **List comprehension:** Concise syntax for creating lists
- **Vectorization:** NumPy operations without explicit loops

Applications:

- Data processing
- Mathematical computations
- Algorithm implementation

(1.7.2) for Loop Examples

Basic for loop syntax:

```
# Sum from 1 to 10
total = 0
for i in range(1, 11):
    total += i
print("Sum from 1 to 10:", total)
# Output: 55

# List element squaring
numbers = [1, 2, 3, 4, 5]
squared = []
for num in numbers:
    squared.append(num ** 2)
print("Squared list:", squared)
# Output: [1, 4, 9, 16, 25]
```

Sum of Odd Numbers

```
print("Calculate sum of odd numbers from 0 to n.")
print("Enter n:")
n = int(input())

val = 0
for i in range(0, n + 1):
    if i % 2 == 1: # Check if odd
        val = val + i

print("Sum of odd numbers up to n:", val)

# Example: n = 100
# Output: Sum of odd numbers up to n: 2500
```

Explanation:

- Loop through 0 to n
- Check if number is odd using modulo operator
- Add odd numbers to running total

(1.7.3) while Loop

Basic while loop syntax:

```
1 while condition:
2     # Code to execute repeatedly
3
4 # Example: Sum from 1 to n
5 print("Calculate sum from 1 to n.")
6 n = int(input("Enter n: "))
7
8 i = 1
9 total = 0
10 while i <= n:
11     total += i
12     i += 1
13
14 print(f"Sum from 1 to {n} is {total}.")
15
16 # Example: n = 100
17 # Output: Sum from 1 to 100 is 5050.
```

(1.8) Conditional Calculations

Overview

Conditional calculation is a computation method that performs different operations or tasks based on specific conditions.

Basic if-else syntax:

```
1 if condition:
2     # Code to execute if condition is True
3 else:
4     # Code to execute if condition is False
5
6 # Example
7 num = int(input("Enter a number: "))
8 if num % 2 == 0:
9     print(f"{num} is even.")
10 else:
11     print(f"{num} is odd.")
```

Multiple Conditions with elif

if-elif-else structure:

```
scores = [95, 82, 67, 58, 89]
grades = []

for score in scores:
    if score >= 90:
        grades.append('A')
    elif score >= 80:
        grades.append('B')
    elif score >= 70:
        grades.append('C')
    else:
        grades.append('D')

print("Scores:", scores)
print("Grades:", grades)
# Output: Scores: [95, 82, 67, 58, 89]
#          Grades: ['A', 'B', 'D', 'D', 'B']
```

Comparison Operators

Operator	Meaning	Example	Result
<code>==</code>	Equal	<code>5 == 5</code>	True
<code>!=</code>	Not equal	<code>5 != 3</code>	True
<code>></code>	Greater than	<code>5 > 3</code>	True
<code>>=</code>	Greater or equal	<code>5 >= 5</code>	True
<code><</code>	Less than	<code>3 < 5</code>	True
<code><=</code>	Less or equal	<code>3 <= 5</code>	True

Usage:

- Conditional statements (if, elif, else)
- While loops
- Boolean expressions

(1.9) Input and Output

Overview

This section explains methods for handling data input and output in Python.

Keyboard Input and Screen Output:

```
# Input from user
name = input("Enter your name: ")
age = int(input("Enter your age: "))

# Output to screen
print(f"Hello, {name}.")
print(f"You are {age} years old.")

# Example:
# Enter your name: Seejo Kim
# Enter your age: 22
# Output: Hello, Seejo Kim.
#         You are 22 years old.
```

File Input and Output

Writing to a file:

```
1 # File writing
2 with open("output.txt", "w") as file:
3     file.write("Hello.\n")
4     file.write("Python file I/O example.\n")
5
6 # File reading
7 with open("output.txt", "r") as file:
8     content = file.read()
9
10 print("File content:\n", content)
11 # Output:
12 # File content:
13 # Hello.
14 # Python file I/O example.
```

File modes: 'w' (write), 'r' (read), 'a' (append)

Saving and loading matrices using NumPy:

```
import numpy as np

# Create matrix
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Created matrix A:")
print(A)

# Save to file
np.savetxt("matrix.txt", A, fmt="%d", delimiter=",")
print("\nMatrix A saved to 'matrix.txt'.")

# Load from file
loaded_A = np.loadtxt("matrix.txt", delimiter=",", dtype=int)
print("\nMatrix A loaded from file:")
print(loaded_A)
```

(1.10) Functions

Overview

Functions are independent blocks that perform specific tasks, used to modularize repetitive work and improve code reusability and readability.

Basic function syntax:

```
def function_name(parameters):  
    # Function body  
    return value  
  
# Example  
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
print("5! =", factorial(5)) # Output: 5! = 120  
print("7! =", factorial(7)) # Output: 7! = 5040
```


Built-in Functions vs User-Defined Functions

Built-in Functions:

- Python provides many built-in functions
- Examples: `len()`, `sum()`, `max()`, `min()`, `print()`
- Can be used directly without definition

User-Defined Functions:

```
# Define function
def f(x):
    return 9 - x * (x - 10)

def fprime(x):
    return -2 * x + 10

# Use functions
print("f(3) =", f(3))          # Output: f(3) = 0
print("f'(3) =", fprime(3))    # Output: f'(3) = 4
```

(1.11) Vectors and Matrices using NumPy

Overview

This section explains how to process vector and matrix operations using NumPy, Python's high-performance numerical computation library.

Key Topics:

- Dot product (inner product)
- Cross product (outer product)
- Distance between vectors
- Matrix multiplication

Advantages of NumPy:

- Vectorized operations (no explicit loops needed)
- High-dimensional array support
- Efficient linear algebra operations

(1.11.1.1) Dot Product

For n-dimensional vectors:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n \quad (1.11.1-1)$$

```
import numpy as np

# Define two vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Calculate dot product
dot_product = np.dot(a, b)
print("Dot product of a and b:", dot_product)
# Output: 32
```

Applications:

- Calculating angle between vectors
- Vector projections
- Similarity measures

(1.11.1.2) Cross Product

Cross product (3D vectors only):

```
# Define two 3D vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Calculate cross product
cross_product = np.cross(a, b)
print("Cross product of a and b:", cross_product)
# Output: [-3  6 -3]
```

Properties:

- Result is perpendicular to both input vectors
- Magnitude represents area of parallelogram
- Only defined for 3D vectors
- Anti-commutative: $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$

(1.11.2) Distance Between Vectors

Euclidean distance for n-dimensional vectors:

$$\|\mathbf{a} - \mathbf{b}\| = \sqrt{\sum_{i=1}^n (a_i - b_i)^2} \quad (1.11.2-1)$$

```
import numpy as np

# Define two vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Calculate Euclidean distance
distance = np.linalg.norm(a - b)

print("Vector a:", a)
print("Vector b:", b)
print("Distance between vectors:", distance)
# Output: 5.196152422706632
```

(1.11.3) Matrix Multiplication

For $m \times n$ matrix **A** and $n \times p$ matrix **B**:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj} \quad (1.11.3-1)$$

```
import numpy as np

# Define matrices
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
C = A @ B # or np.dot(A, B)
print("Matrix product:\n", C)

# Output:
# [[19 22]
#  [43 50]]
```

Note: Use `@` or `np.dot()` for matrix multiplication, not `*` (element-wise)!

(1.12) Object-Oriented Programming (OOP)

Overview

OOP is a programming paradigm that models data as objects and designs programs through interaction between objects.

Key Concepts:

Encapsulation Bundling data and methods together, implementing information hiding

Inheritance Child classes inherit characteristics from parent classes, increasing code reusability

Polymorphism Using the same method name to perform different operations for different objects

Benefits:

- Code reusability
- Easier maintenance
- Better organization

OOP Example

```
# Define class
class Animal:
    def __init__(self, name): # Constructor
        self.name = name

    def speak(self):
        print(f"{self.name} is making a sound.")

# Inheritance
class Dog(Animal):
    def speak(self): # Method overriding (polymorphism)
        print(f"{self.name} says Woof!")

# Create object and use
dog = Dog("Buddy")
dog.speak()
# Output: Buddy says Woof!
```


Class methods operate at the class level:

```
class Example:
    class_variable = 0

    def __init__(self, value):
        self.instance_variable = value

    @classmethod
    def increment_class_variable(cls, amount):
        cls.class_variable += amount
        print(f"Class variable updated to {cls.class_variable}")

    @classmethod
    def create_instance(cls, value):
        return cls(value)

# Usage
Example.increment_class_variable(10)
# Output: Class variable updated to 10

instance = Example.create_instance(20)
print(instance.instance_variable) # Output: 20
```

(1.13) Graph Visualization using Matplotlib

Overview

This section explains how to express data as graphs using Matplotlib, Python's data visualization library.

Supported Graph Types:

- Line graphs
- Bar charts
- Histograms
- Scatter plots
- 3D surface plots

Key Components:

- Title, axis labels, legends
- Plot styles and colors
- Subplots for multiple visualizations

2D Line Graph Example

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
x = np.linspace(-10, 10, 500)
y = x**2

# Create graph
plt.figure(figsize=(10, 6))
plt.plot(x, y, label='f(x) = x^2', color='blue', linewidth=2)

# Add annotations
plt.title('Line Graph of f(x) = x^2', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('f(x)', fontsize=12)
plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
plt.axvline(0, color='black', linestyle='--', linewidth=0.8)
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(fontsize=12)

plt.show()
```

2D Scatter Plot

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
np.random.seed(42)
x = np.random.rand(100) * 10
y = 2 * x + np.random.normal(0, 3, 100)

# Create scatter plot
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='blue', alpha=0.7, label='Data points')

# Add annotations
plt.title('2D Scatter Plot', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(fontsize=12)

plt.show()
```

3D Surface Plot

```
1 from mpl_toolkits.mplot3d import Axes3D
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Prepare data
6 x = np.linspace(-5, 5, 50)
7 y = np.linspace(-5, 5, 50)
8 X, Y = np.meshgrid(x, y)
9 Z = np.sin(np.sqrt(X**2 + Y**2))
10
11 # Create 3D graph
12 fig = plt.figure()
13 ax = fig.add_subplot(111, projection="3d")
14 ax.plot_surface(X, Y, Z, cmap="viridis")
15
16 ax.set_title("3D surface")
17 ax.set_xlabel("X axis")
18 ax.set_ylabel("Y axis")
19 ax.set_zlabel("Z axis")
20
21 plt.show()
```

(1.14) Exercise Problems

Problem 1: Write code using List Comprehension to calculate squares from 1 to 10 and explain.

Problem 2: Python variables don't need to specify data types, but write a program that uses the `type()` function to check variable data types and explain.

Problem 3: Create two 3×3 matrices and write a program that calculates the sum and product of the matrices and explain.

Problem 4: Write a program that calculates $\sin(\pi/4)$ using NumPy's built-in function and explain.

Exercise Problems (Continued)

Problem 5: Write a program that calculates the sum of integers from 1 to 100 and explain.

Problem 6: Write a program that takes two numbers as input and outputs the larger value and explain.

Problem 7: Write a program that takes name and age from user input and outputs "Hello, [name]! You are [age] years old." and explain.

Problem 8: Write a program using user-defined functions and Python built-in functions (`len`, `sum`, `max`) to calculate the length, sum, and maximum value of a list.

Exercise Problems (Continued)

Problem 9: Create two 3-dimensional vectors and write a program that calculates the dot product of the vectors.

Problem 10: Write a program that plots the graph of $y = \sin(x)$ from 0 to 10.

Summary: Key Takeaways

- ① **Python Features:** Platform independence, dynamic typing, interpreter-based execution
- ② **Data Structures:** Lists, tuples, dictionaries each have specific use cases
- ③ **NumPy:** Essential for high-performance numerical computing
- ④ **Control Flow:** Mastery of loops and conditionals is fundamental
- ⑤ **Functions:** Improve code reusability and organization
- ⑥ **OOP:** Enables systematic and efficient problem solving
- ⑦ **Visualization:** Matplotlib makes data analysis intuitive

Python in Numerical Analysis

Why Python is Ideal for Numerical Analysis:

- **Simple syntax:** Focus on algorithms, not syntax
- **Rich ecosystem:** NumPy, SciPy, Matplotlib, pandas
- **Rapid prototyping:** Test ideas quickly
- **Community support:** Extensive documentation and examples
- **Integration:** Easily connects with other tools and languages

Applications in Engineering:

- Solving differential equations
- Matrix computations
- Optimization problems
- Signal processing
- Data analysis and visualization

Writing Good Python Code:

- ➊ **Use meaningful variable names:** Make code self-documenting
- ➋ **Follow PEP 8:** Python's style guide
- ➌ **Write docstrings:** Document functions and classes
- ➍ **Use built-in functions:** They're optimized
- ➎ **Leverage NumPy:** Avoid loops for array operations
- ➏ **Test your code:** Verify correctness with examples
- ➐ **Comment wisely:** Explain why, not what

Remember: "Readability counts" - The Zen of Python

Resources for Further Learning

Official Documentation:

- Python: <https://docs.python.org>
- NumPy: <https://numpy.org/doc>
- Matplotlib: <https://matplotlib.org/stable/contents.html>

Online Resources:

- Python Tutorial: <https://www.python.org/about/gettingstarted>
- NumPy Tutorial: <https://numpy.org/learn>
- Stack Overflow: For troubleshooting
- GitHub: For code examples

Books:

- "Python for Data Analysis" by Wes McKinney
- "Numerical Python" by Robert Johansson
- "Python Crash Course" by Eric Matthes

Thank You!

Questions?

Prof. See Jo Kim
sjkim1@gknu.ac.kr
Gyeongkuk National University